

Plataforma AME

Estado del proyecto y trabajo pendiente · 28/07/2026

Este documento resume qué está construido, qué falta y en qué orden conviene encararlo. El criterio de prioridad no es técnico sino de negocio: el problema declarado del centro es la **adquisición de clientas** y la **baja conversión a servicios**, con la venta de productos como fuente principal de ingresos. Todo lo que no acerque la app a manos de clientas reales queda por detrás.

Estado actual

La plataforma cubre el ciclo de vida de la clienta de punta a punta. Lo que sigue ya está construido, probado y funcionando:

- **Backend del cliente:** autenticación JWT propia, aislada de la del staff; perfil, rutina, turnos, catálogo público y registro de token de push. Todo con alcance por vinculación y serializers curados. 92 tests en verde.
- **App mobile:** alta de cuenta y login, inicio con próximo turno real, Mi rutina, perfil, listado de turnos con cancelación, reserva en tres pasos con política de días, y catálogo de tratamientos con ficha de detalle.
- **CRM del staff:** editor de rutinas, detector y fusión de fichas duplicadas, control de qué servicios se pueden reservar desde la app y carga del contenido de la ficha.
- **Datos:** vinculación multi-centro, teléfonos normalizados y los campos necesarios para el motor de recompra.

La **ficha de servicio** se construyó en esta iteración: cada tratamiento tiene su pantalla con descripción, beneficios y enlace de video, el catálogo completo es navegable y el botón de reservar entra al flujo con el tratamiento ya elegido. Se hizo fuera del orden sugerido —era el punto 7 de 9—, así que los puntos anteriores siguen abiertos. La pantalla se ve vacía hasta que alguien escriba el texto de cada tratamiento: eso es ahora trabajo del centro, no de programación, y es lo que decide si convierte o no.

El defecto de privacidad en el cache de la app (una cuenta podía ver datos de la anterior tras cerrar sesión) fue corregido y ya no figura como pendiente.

1.1 Distribución de la app

Es el verdadero cuello de botella. Hoy la app **solo corre en Expo Go sobre la máquina de desarrollo**. No hay nada resuelto sobre cómo llega al teléfono de una clienta.

- Configurar EAS Build y generar los binarios.
- Cuenta de desarrollador de Apple (anual) y de Google Play (pago único).
- Publicación y revisión en ambas tiendas: es tiempo de calendario, no de código.
- Definir el canal de actualizaciones (EAS Update permite corregir sin pasar por revisión, hoy no está configurado).
- Habilitar pruebas en dispositivo físico: la app apunta a localhost, falta la IP de red local en ALLOWED_HOSTS y CORS.

Atajo recomendado: resolver primero el punto del dispositivo físico y poner la app en manos de dos o tres clientas con Expo Go. Da retorno real en días, sin esperar a las tiendas.

1.2 Notificaciones push

El endpoint de registro de token ya existe; falta integrar expo-notifications en la app y disparar los envíos desde el backend. Con la integración de WhatsApp caída, **push es el único canal** para llegar a la clienta.

- Confirmación del turno cuando el staff lo aprueba desde el CRM.
- Recordatorios previos al turno.
- Es requisito de otras funcionalidades: sin recordatorio no hay hábito, y sin push la confirmación del staff no llega hasta que la persona abra la app.

1.3 Onboarding de producción

Decisión abierta. Hoy la vinculación exige que el staff genere un código y la clienta lo tipee: sirve para probar, pero es fricción alta para lanzamiento. Las alternativas evaluadas fueron enlace mágico o QR, verificación por SMS (bloqueada hoy) y auto-registro puro. Hay que definirlo antes de publicar.

1.4 Almacenamiento de imágenes

Las imágenes se guardan en el disco local del servidor. En un hosting como Render ese disco es efímero: **cada despliegue borra todo lo que se haya subido**. Hay que resolverlo antes de que el centro cargue fotos, porque migrar imágenes ya cargadas cuesta mucho más que empezar bien.

- Afecta a las fotos de producto, que la app ya muestra en Mi rutina.
- En el mismo lugar están las fotos de antes y después de las clientas: son datos de salud de personas identificables y no pueden quedar en direcciones públicas. Hay que separar un espacio público del catálogo de uno privado con acceso firmado.
- El CRM no tiene ningún formulario de carga de archivos: hoy las fotos solo se suben desde el panel de administración de Django.

2.1 Frecuencia por paso en la rutina

El modelo de la rutina no tiene frecuencia: todo paso es implícitamente diario, así que no se puede prescribir "exfoliar dos veces por semana". Es un problema clínico, no cosmético, y además bloquea cualquier sistema de seguimiento o rachas.

2.2 Frescura de los datos entre web y app

La app no se entera sola de los cambios hechos en el CRM: solo refresca al abrirla, al deslizar para actualizar o después de una acción propia. Falta conectar el estado de la aplicación para que refresque al volver del segundo plano. No es un defecto de privacidad sino de frescura, y se resuelve junto con push.

2.3 Otros pendientes conocidos

- Migrar y eliminar los campos de texto legados de la rutina.
- El botón "Comprar" de la rutina es un placeholder sin acción.
- El producto asociado a un paso de rutina acepta cualquier identificador: falta restringirlo al centro de la clienta.
- No existe reprogramar un turno; solo cancelar y reservar de nuevo.
- El soporte multi-centro está modelado pero la app usa siempre la primera vinculación.
- Tres errores de tipos preexistentes en la barra de navegación (no afectan runtime).
- **No hay instrumentación de ningún tipo:** al lanzar no se va a poder responder qué se usa y qué no. Con la ficha recién construida esto duele más: no hay forma de saber si alguien la abre.

2.4 Pantallas incompletas

- Promos: es un placeholder. Es el lugar natural del programa de puntos y beneficios.
- Detalle de producto: no existe. Es el equivalente de la ficha de servicio del lado de los productos, que son la fuente principal de ingresos.

3.1 Video reproducible dentro de la app

La ficha ya guarda un enlace de video, y hoy ese enlace abre la aplicación de origen. Que el video se reproduzca sin salir de la app depende de dónde esté alojado:

- **YouTube:** se puede embeber. Es medio día de trabajo y no tiene costo de infraestructura. El video puede estar en modo "no listado" para que no aparezca en búsquedas.
- **Instagram:** no se puede. La plataforma no permite reproducir un reel fuera de su dominio, así que ese caso se queda con el enlace que abre Instagram.
- **Archivo propio:** la mejor experiencia y la más cara. Requiere el almacenamiento del punto 1.4, subida desde el CRM y conversión del video para que no consuma los datos móviles de la clienta.

Decisión abierta: dónde va a subir los videos el centro. Si es YouTube conviene hacerlo; si es Instagram, no hay nada que construir.

3.2 Cruce de servicio con producto

La ficha se construyó sin productos relacionados, por decisión, para no bloquear la primera versión. Es la pieza que la conecta con la fuente principal de ingresos: quien mira un tratamiento debería ver qué productos lo acompañan, y quien compra un producto debería ver a qué tratamiento lleva. Requiere una relación nueva entre servicio y producto, y una interfaz en el CRM para asociarlos.

3.3 Rachas de rutina de skincare

Registro diario de la rutina con racha, al estilo de las apps de hábitos. El valor no es el estímulo sino **el dato de consumo**: saber a qué velocidad se gasta cada producto habilita la recompra predictiva, que sí toca la facturación.

- Depende de push: sin recordatorio, nadie registra.
- Depende de la frecuencia por paso (punto 2.1): premiar el uso diario de ácidos o retinol sería dañino.
- La racha debe ser tolerante; romperla no puede generar abandono.
- Sin tablas comparativas entre clientas: son pocas y es información sensible.
- Validar con una versión mínima antes de invertir: si nadie registra el segundo día, no se construye el resto.

Definido explícitamente como lo último del plan. La factibilidad ya fue analizada y no presenta obstáculos técnicos serios, pero arrastra decisiones de negocio que no están tomadas y que conviene no apurar.

4.1 Enfoque técnico previsto

- Integración con la API directa de Tienda Nube, sin intermediarios.
- Un módulo nuevo en el backend Django que reciba los webhooks de la tienda (pedidos creados, pagados, cancelados).
- Mapeo entre el catálogo de Tienda Nube y el inventario propio.
- Impacto en el sistema financiero: los pedidos online deberían generar sus transacciones igual que las ventas presenciales, para no partir la contabilidad.

4.2 Decisiones abiertas

Estas dos preguntas hay que responderlas antes de escribir código, porque cambian la arquitectura por completo:

- **¿El stock físico y el online son el mismo pool?** Si lo son, hace falta sincronización en ambos sentidos y resolver qué pasa cuando dos ventas simultáneas compiten por la última unidad. Si no lo son, hay que definir cómo se reparte y quién lo administra.
- **¿Cuál es el rol de la "Plataforma 2"?** Sin eso definido no se sabe si este módulo vive en el sistema actual o en otro lado.

4.3 Oportunidad y dependencias

Es la funcionalidad con vínculo más directo con la fuente principal de ingresos: permitiría cerrar el circuito entre la rutina de cuidado, el aviso de recompra y la compra efectiva, sin que la clienta tenga que ir al centro. Por eso, aunque vaya última, conviene que las piezas previas se construyan pensando en ella: el motor de recompra, la ficha de producto y el registro de consumo alimentan directamente este canal.

5. Orden sugerido

#	Trabajo	Por qué va acá
1	Pruebas en dispositivo físico	Desbloquea probar con clientas reales en días, sin costo ni espera.
2	Notificaciones push	Único canal disponible. Habilita confirmaciones y recordatorios.
3	Contenido de las fichas	Sin texto la pantalla se ve vacía. Lo hace el centro, en paralelo.
4	Distribución (EAS + tiendas)	Tiempo de calendario: conviene empezar el trámite cuanto antes.
5	Onboarding de producción	Decisión de producto que bloquea el lanzamiento.
6	Almacenamiento de imágenes	Antes de que se suban fotos; migrarlas después cuesta más.
7	Instrumentación básica	Sin medir, no se puede decidir qué construir después.
8	Frecuencia por paso en la rutina	Corrige un problema clínico y habilita el seguimiento.
9	Cruce de servicio con producto	Conecta la ficha con la venta de productos.
10	Rachas y recompra predictiva	Depende de los puntos 2 y 8. Genera el dato de consumo.
11	E-commerce (Tienda Nube)	Último por decisión propia; requiere definiciones de negocio previas.

La ficha de servicio salió de esta lista: está construida. El resto del diagnóstico no cambió: se agregó funcionalidad mientras los puntos que realmente bloquean el lanzamiento siguen abiertos. Hoy el límite del negocio no es la falta de features, es que la aplicación todavía no está en manos de nadie.